

01_PROCESSES AND THREADS

1. PROCESSES: fork(), wait()

Processo: entità dinamica caricata sulla memoria RAM generata da un programma. Più precisamente è una sequenza di attività (**task**) controllate da un **program scheduler** che tipicamente si svolge su un processore sotto la gestione o supervisione del sistema operativo

La maggior parte dei requisiti che un OS deve soddisfare possono essere espressi:

- Interleaved execution
- Resource allocation and policies
- User creation of processes and inter-process communication

Un processo è composto da:

- Program code (possibly shared)
- Un insieme di dati
- Un insieme di attributi che descrivono lo stato del processo durante l'esecuzione

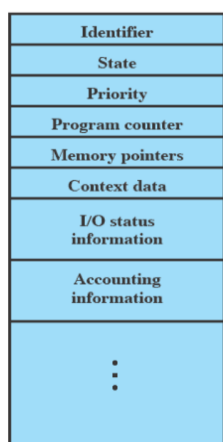
Mentre il processo è in esecuzione ha degli elementi:

- Identifier
- State
- Priority
- Program counter
- Memory pointers
- Context data
- I/O status information
- Accounting information

Tutte queste informazioni sono raccolte all'interno del **Process Control Block (PCB)**, che contiene gli elementi del processo.

Viene creato e gestito dall'OS; i PCB permettono all'OS di poter gestire più processi.

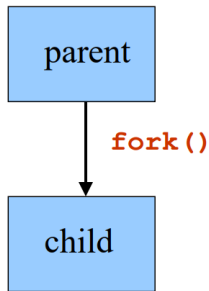
PCB:



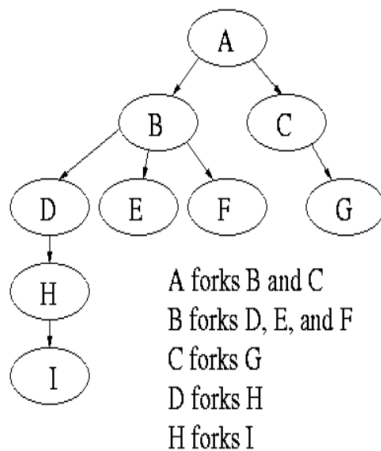
I comandi principali per gestire i processi sono: fork(), wait(), exit()

Creazione di un nuovo processo:

un processo padre genera un processo figlio tramite la **fork()**. Padre e figlio vengono eseguiti concorrentemente. Il processo figlio è un duplicato del processo padre.



Dopo una fork sia padre che figlio continuano ad essere in esecuzione, entrambi possono fare altre fork. Ne risulta un **process tree (albero dei processi)**



La radice dell'albero è un processo speciale creato dall'OS durante l'avvio.

Un processo può decidere di aspettare la terminazione dei processi figli con la **wait()**.

Ad esempio se il processo C chiama una system call wait si arresta fino a quando non finisce il processo G.

Bootstrapping: quando un computer viene acceso o riavviato, dev'esserci un programma iniziale che avvia l'esecuzione del sistema

Dopo una fork il processo corrente si divide in 2: **parent** e **child**.

La fork restituisce:

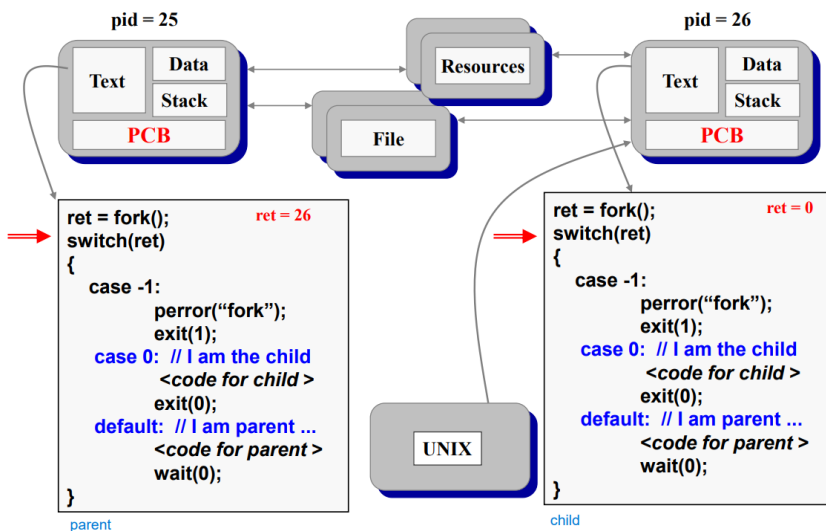
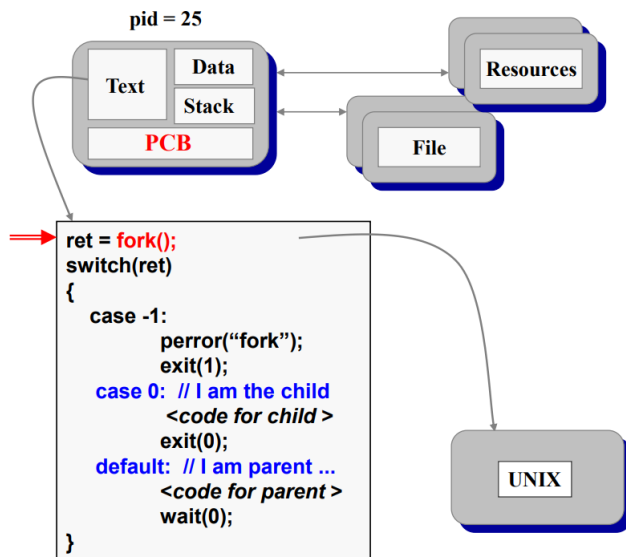
- 1 se non ha avuto successo
- 0 al processo figlio
- child identifier al processo padre

Il processo figlio eredita dal padre:

- Una copia identica della memoria
- Registri CPU
- Tutti i file che sono stati aperti dal padre

L'esecuzione riprende dall'istruzione successiva alla fork.

Il PCB del figlio è una copia di quello del padre



Se la `fork()` viene usata per creare un nuovo processo che deve eseguire un programma allora usiamo il comando `exec()`, l'OS rimpiazza l'immagine del processo corrente (text, data, stack) con l'immagine di un nuovo processo. Il nuovo processo deve avere una funzione `main()`.

Per terminare un'esecuzione un figlio potrebbe chiamare la `exit(status)`.

Questa system call:

- salva il risultato (status)
- esegue tutte le funzioni specificate con `atexit(fun)` e `on_exit(fun)`
- vengono scaricati gli stream con `fflush()`
- chiude tutti i file aperti e connessioni (non quelli condivisi)
- chiama la `_exit(status)`

La **_exit(status)**:

- salva il risultato (status)
- dealloca la memoria
- se il processo ha dei figli li assegna a **init** (processo iniziale dell'OS, radice dell'albero dei processi)
- controlla il padre:
se il padre è vivo mantiene il valore del risultato finché il padre non lo richiede con la wait. In questo caso il processo figlio non muore veramente, ma entra nello stato **zombie**.
Se invece il padre è morto allora il figlio muore

Per aspettare la terminazione di un figlio qualunque: **wait()**. Restituisce il pid del processo figlio terminato (-1 se non esistono figli).

Per aspettare la terminazione di un figlio specifico: **waitpid()**.

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

In python:

- Process creation is managed by operating system
- Python must be able to ask the OS to do it
 - import os
- Commands are similar
 - pid = os.fork()
 - os.wait()
 - os.waitpid()
 - os.exit()
 - os._exit()

```
import os

ret = os.fork()
if ret==0:
    # I am the child
    <code for child>
    os._exit(0)
else:
    # I am parent ...
    <code for parent >
    os.wait(0);

< ... >
```

2. THREADS: RESOURCE OWNERSHIP AND EXECUTION

Un processo ha due caratteristiche:

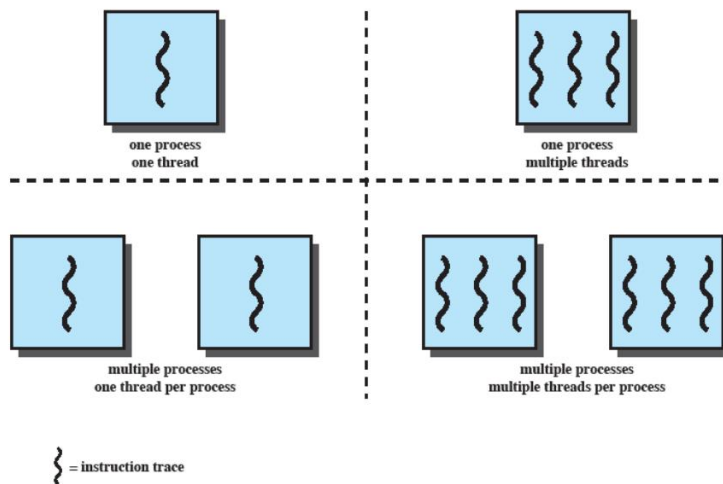
- Scheduling/execution: segue un execution path che potrebbe essere intervallato con altri processi
- Resource ownership: include un virtual address space per contenere l'immagine di processo

Queste caratteristiche sono trattate indipendentemente dall'OS

Thread: unit of dispatching (lightweight process)

Task: unit of resource ownership

Multithreading: capacità di un OS di supportare molteplici concorrenti execution path entro un singolo processo.



Approcci Single-Thread (un thread per processo):

MS-DOS (Microsoft) supports a single user process and a single thread

Some UNIX support multiple user processes but only support one thread per process.

Multithreading:

Spesso un Java run-time environment è un singolo processo con molteplici thread

In Windows e altre moderne versioni UNIX possiamo trovare molteplici processi e thread

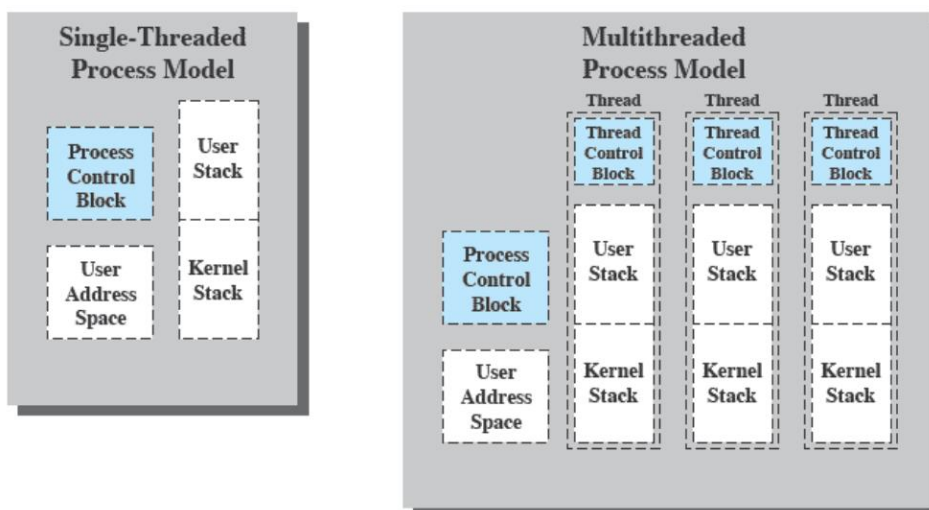
In un Multithreaded OS un virtual address space contiene l'immagine del processo.

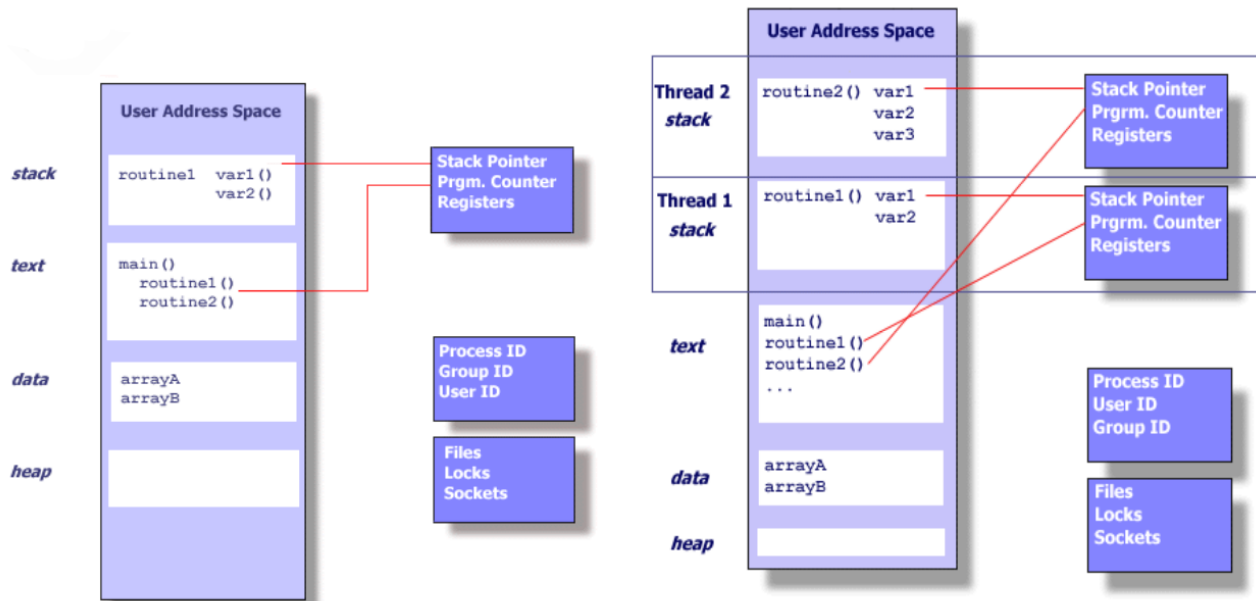
Si ha accesso protetto a processori, altri processi, file e risorse I/O

Ogni thread ha:

- Un execution state (running, ready, ...)
- Saved thread context when not running
- Una execution stack
- Della memoria statica per ciascun thread per variabili locali
- Accesso a memoria e risorse del proprio processo (condiviso con ogni thread del processo)

Un thread si può vedere come un program counter indipendente che opera dentro ad un processo.





Vantaggi dell'utilizzo di thread:

- Ci vuole meno tempo per creare o terminare un nuovo thread rispetto a un processo
- Saltare fra due thread è molto più rapido
- I thread possono comunicare tramite l'area di memoria condivisa senza invocare il kernel

I thread si possono usare per:

- Foreground and background work
- Asynchronous processing
- Speed of execution
- Modular program structure

Alcune azioni possono avere conseguenze su tutti i thread di un processo. Ad esempio la terminazione di un processo con la `exit` provoca la terminazione di ogni thread.

I thread hanno stati di esecuzione e potrebbero aver bisogno di sincronizzarsi con altri thread (simile ai processi).

Stati di un thread: running, ready, blocked

Operazioni che cambiano lo stato di un thread:

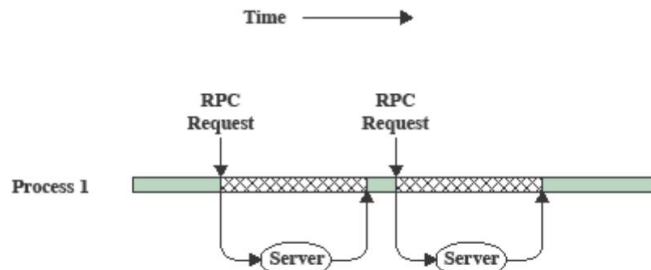
- `Spawn` (another thread), genera un nuovo thread
- `Block`, serve a bloccare un thread in attesa di qualcosa (problema: can blocking a thread result in blocking some other thread, or even the whole process?)
- `Unblock`, sblocca un thread
- `Finish` (thread), dealloca register context e stack. Porta da qualsiasi stato alla terminazione

Vediamo un esempio di utilizzo di thread: **Remote Procedure Call (RPC)**

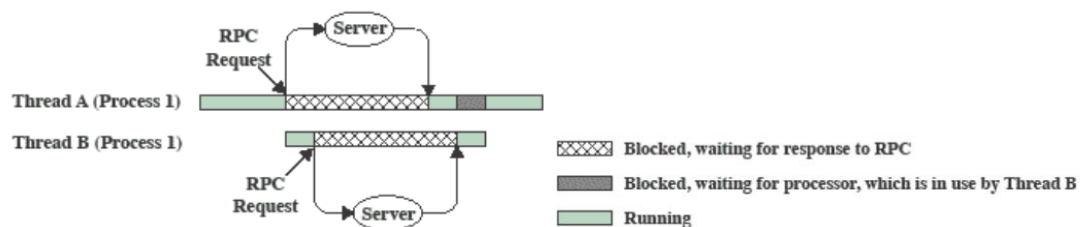
consideriamo un programma che esegue due remote procedure calls (RPC) a due diversi host, per ottenere un risultato combinato.

Ad esempio un client che invia due richieste a due server diversi, aspetta la risposta da entrambi e formula un risultato combinato a partire dalle due risposte ottenute.

RPC con single thread:

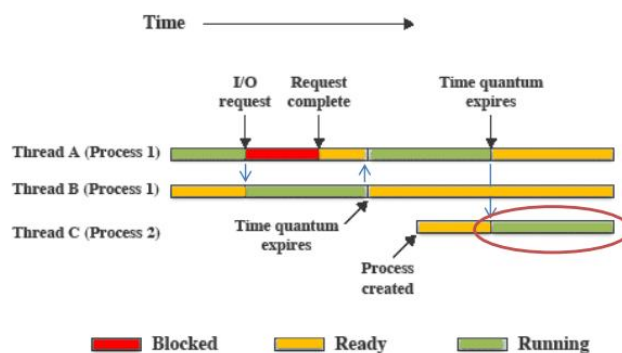


RPC che usa un thread per server:



Il thread B inizia e invia la propria RPC non appena il thread A manda la sua RPC.

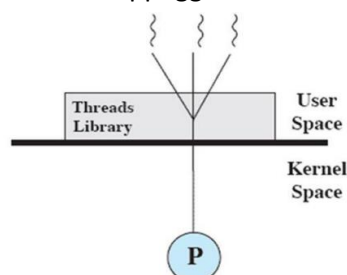
Multithreading su un unico processore:



Categorie di implementazioni di thread:

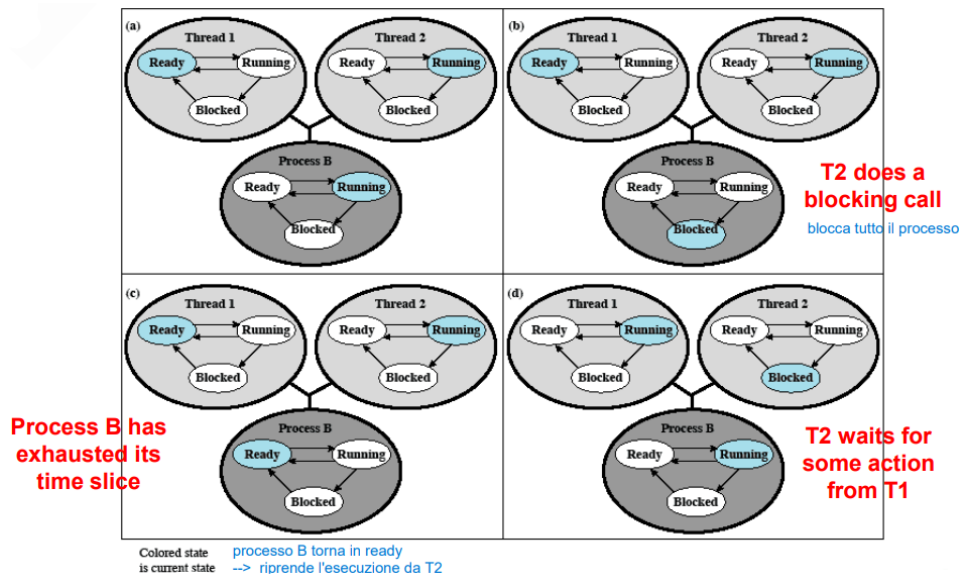
- **User Level Thread (ULT):**

Tutta la gestione dei thread è svolta dall'applicazione. Il kernel non è consapevole della presenza di thread. Il tutto si appoggia su delle librerie.



Rapporto fra ULT threads e stati del processo: se uno dei thread si blocca, ad esempio per richiedere un qualcosa da I/O, allora il processo al quale appartiene invierà questa richiesta al kernel, quindi l'intero processo rimane bloccato (non vanno in esecuzione altri thread che potrebbero invece continuare).

es.



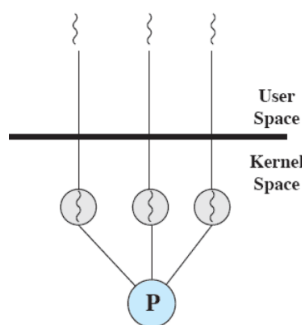
(a) Il thread T2 del processo B è in esecuzione, T1 è in ready.

(b) T2 fa una blocking call, fa cioè una richiesta → il processo B si blocca e passa la richiesta al kernel

(c) ricevuta la risposta il processo B torna in stato ready e riprende l'esecuzione da dove si era fermato nel thread T2

(d) T2 entra in blocked per far passare a running T1

- **Kernel level Thread (KLT)**, anche chiamato kernel-supported threads o lightweight processes: Il kernel mantiene context information per il processo e per i thread. È quindi l'OS a gestire i thread come tanti piccoli processi, non l'applicazione. Lo scheduling è fatto sulla base dei thread.



Vantaggi ULT:

- Possiamo definire una application-specific thread scheduling, indipendentemente dalle politiche di gestione dei processi del sistema operativo
- Thread switch non richiede kernel privilege/switch to kernel mode
- Possono runnare su qualsiasi OS: l'implementazione è fatta attraverso una thread library a user level

Svantaggi ULT:

- A blocking systems call executed by a thread blocks all threads of the process
- Pure ULTs does not take full advantage of multiprocessors/multicores architectures

Vantaggi KLT:

- Il kernel può contemporaneamente fare scheduling di multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process
- Kernel routines themselves can be multithreaded

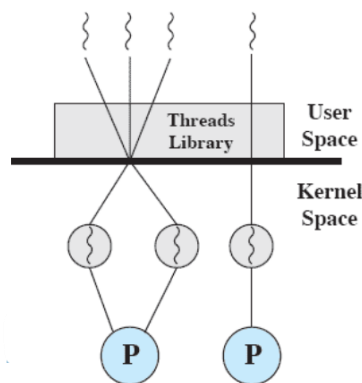
Svantaggi KLT:

- The transfer of control from one thread to another within the same process requires a mode switch to the kernel

Esistono degli approcci combinati:

La creazione di thread viene fatta nell'user space. Bulk of scheduling and synchronization of threads done within the application.

u ULTs are mapped onto k KLTs (k=u in Solaris).



Possibili arrangiamenti di thread e processi:

Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

3. CASE STUDY: Pthreads

PThreads (POSIX Threads): tipi di dato C, definite in pthread.h

Il motivo principale per cui si hanno i Pthread è per migliorare le performance di un programma.

Un Pthread può essere creato con molto meno OS overhead, è necessario eseguire meno system resources.

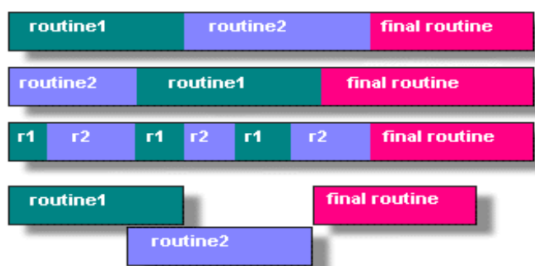
È più efficiente di usare la fork().

Designing Threaded Programs as in Parallel Programming:

Per prendere vantaggio da PThreads (POSIX Threads), un programma dovrebbe essere organizzato in discrete, indipendenti task, che possono essere eseguite concorrentemente

Ad esempio:

if routine1 and routine2 can be interchanged, interleaved and/or overlapped in real time, they are candidates for threading.



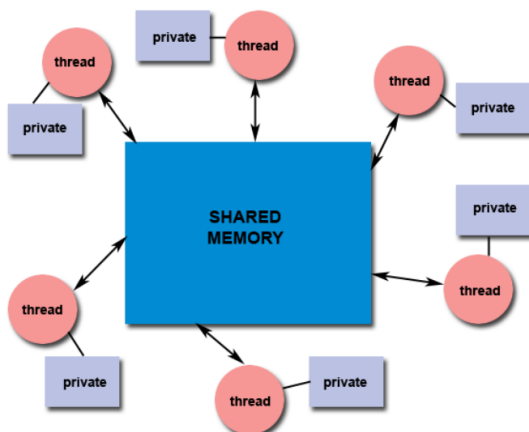
Modelli per threaded programs:

- **Manager/worker:** A manager thread assigns work to other threads, the workers. Manager handles input and hands out the work to the other tasks
- **Pipeline:** A task is broken into a series of suboperations, each handled in series, but concurrently, by a different thread

Modello Shared-memory:

tutti i processi hanno accesso alla stessa memoria globale condivisa. I thread hanno anche il proprio stack privato.

I programmatori sono responsabili per la sincronizzazione degli accessi alla memoria globale condivisa (soprattutto in scrittura). Se un thread A e un thread B modificano la stessa parte di memoria potrebbe esserci un problema (A pensa che ci sia scritta la cosa che ci ha scritto quando magari invece ci sta la cosa che ci ha scritto B subito dopo).



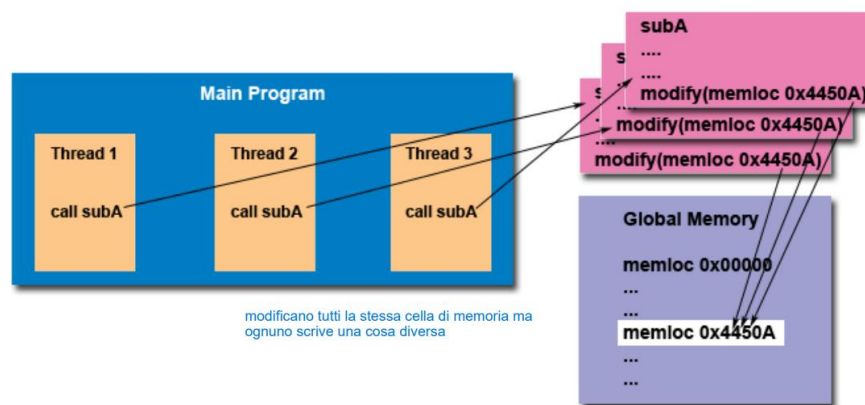
Thread Safety: A code is thread-safe when multiple threads can execute it simultaneously without unintended interactions (senza creare clobbering shared data e senza creare race conditions, ossia il valore finale lo scrive l'ultimo thread che arriva in quella cella di memoria).

es. Si abbia un'applicazione che crea svariati thread, ognuno dei quali fa una call verso la stessa library routine

The library routine accesses/modifies a global structure or location in memory.

As each thread calls this routine, it is possible that they may try to modify this structure/location at the same time.

If the routine does not employ some sort of synchronization mechanism to prevent data corruption, then it is not thread-safe.



Il main() di un programma comprende un singolo thread di default.

pthread_create(): crea un nuovo thread e lo rende eseguibile

Il numero massimo di thread che un processo può creare dipende dall'implementazione.

Una volta creati, i thread sono "peers", e possono creare altri thread a loro volta

Per terminare un thread ci sono diversi modi:

- The thread is complete, i.e., the function it started with reaches a return statement
- **pthread_exit():** is called once a thread has completed its work and it is no longer required to exist
- **pthread_cancel():** termina un thread da un altro thread. Non lo useremo mai.
- **exit():** termina l'intero processo insieme al thread
- The main terminates without executing pthread_exit()

Se il main thread termina prima di ogni altro thread, gli altri thread continueranno l'esecuzione se per terminare il main era stato usato pthread_exit(), o se su di loro era stato invocato pthread_detach().

La pthread_exit() non libera le risorse (ad esempio file aperti), è quindi necessario chiuderle esplicitamente.

es.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_THREADS 5

void* printHello(void *arg) {
    int threadID = *(int*) arg;
    printf("Hey! It's me, thread #%d!\n",
        threadID);
    free(arg);
    pthread_exit(NULL);
}

int main (int argc, char *argv[]) {
    pthread_t threads[NUM_THREADS];
    int ret, t;
    for (t=0; t<NUM_THREADS; t++) {
        printf("In main: creating thread %d\n", t);
        int *arg = malloc(sizeof(int));
        *arg = t;
        ret = pthread_create(&threads[t], NULL,
            printHello, (void*)arg); crea un thread
        if (ret != 0) { funzione che deve eseguire argomenti della funzione
            printf("ERROR: code %d\n", ret); exit(-1);
        } Se uno dei thread non funziona, termina tutto
    }
    pthread_exit(NULL);
}
```

Un possibile risultato d'esecuzione:

```
In main: creating thread 0
In main: creating thread 1
Hey! It's me, thread #0!
In main: creating thread 2
Hey! It's me, thread #2!
Hey! It's me, thread #1!
In main: creating thread 3
In main: creating thread 4
Hey! It's me, thread #3!
Hey! It's me, thread #4!
```

es.

```
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 4

void *hello (void *arg) {
    printf("Hello Thread\n");
}

main() {
    pthread_t tid[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], NULL, hello, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL); aspetta un thread specifico
}
```

Thread in Java:

si usano come una sottoclasse di Thread (o come una classe che implementa runnable)

Thread ha un metodo run() che la sottoclasse deve ridefinire.

Si crea un nuovo thread con new()

Si esegue un thread con start() (che chiama run())

Un thread può attendere un altro thread con join()

Thread in Python:

la Python standard library offre threading, che contiene la maggioranza delle primitive necessarie

Si crea un nuovo thread col metodo Thread()

Si esegue un thread con start() (che esegue la funzione associata)

Un thread può attendere un altro thread con join()

4. SYMMETRIC MULTIPROCESSING (SMP)

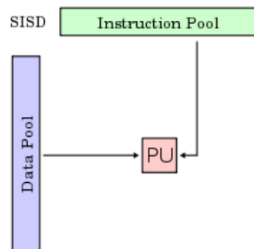
Tradizionalmente il computer è visto come una macchina sequenziale. Un processore esegue istruzioni una alla volta, ogni istruzione è una sequenza di operazioni

Approcci molto usati per il parallelismo:

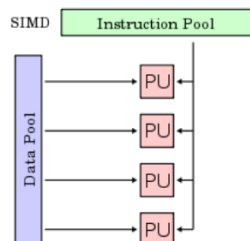
- Symmetric MultiProcessors (SMP)
- Clusters

Categorie di computer system:

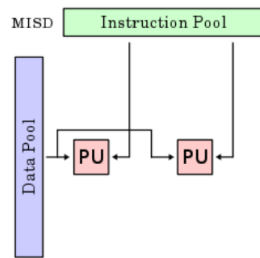
- **Single Instruction Single Data (SISD)**, single processor executes a single instruction stream to operate on data stored in a single memory



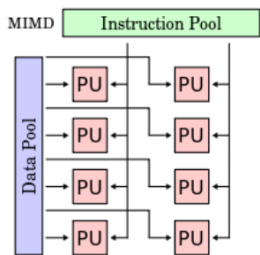
- **Single Instruction Multiple Data (SIMD)**, each instruction is executed on a different set of data by the different processors



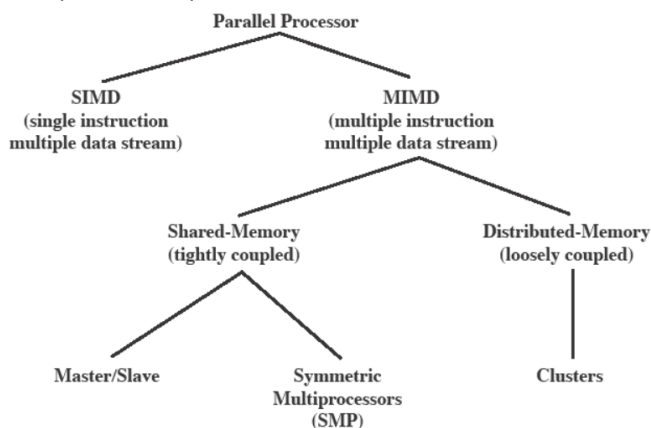
- **Multiple Instruction Single Data (MISD) stream**, a sequence of data is transmitted to a set of processors, each executing a different instruction sequence



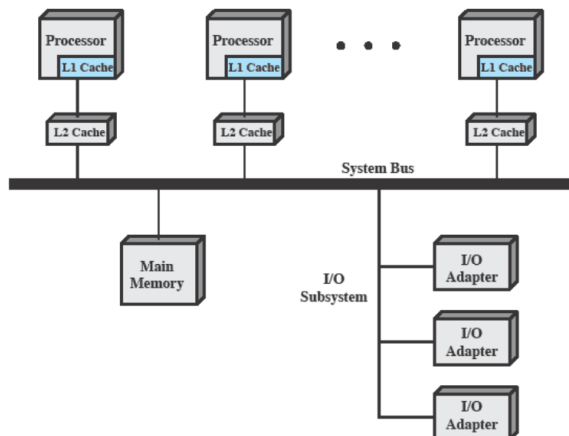
- **Multiple Instruction Multiple Data (MIMD)**, a set of processors simultaneously execute different instruction sequences on different data sets



Architetture processori paralleli:



Organizzazione tipica SMP:



Non so dove il mio dato venga processato.

Considerando Multiprocessor OS Design, le questioni principali di design includono:

- Simultaneous concurrent processes or threads
- Scheduling
- Synchronization
- Memory Management
- Reliability and Fault Tolerance